

THE BARBER

Documentação Técnica e Funcional

Sistema web de gestão e atendimento para barbearia

Projeto	The Barber
Versão documentada	1.0.0
Base tecnológica	Node.js, Express, Sequelize, MySQL e Handlebars
Fonte de verdade	Código-fonte fornecido pelo usuário
Data	13 de julho de 2026

Documento elaborado a partir da inspeção estática do projeto. Funcionalidades planejadas são explicitamente separadas das funcionalidades implementadas.

Documentação preparada por Manus AI

Sumário

Atualize este campo no Microsoft Word para exibir o sumário.

Observação: o sumário é um campo automático do Word e será atualizado ao abrir o documento. Se necessário, selecione-o e use “Atualizar Campo”.

*Esta documentação descreve o estado efetivamente implementado no código-fonte. Funcionalidades presentes apenas em telas, comentários ou planos são identificadas como **parciais**, **não implementadas** ou **planejadas**.*

1. Controle do documento

Item	Descrição
Finalidade	Consolidar a documentação funcional, técnica, operacional e evolutiva do projeto The Barber.
Público-alvo	Equipe de desenvolvimento, docentes e avaliadores, responsáveis por manutenção e futuros colaboradores.
Escopo	Aplicação Node.js, rotas Express, autenticação JWT, views Handlebars, modelos Sequelize e banco MySQL.
Fora do escopo	Infraestrutura de produção, integrações externas e funcionalidades ainda inexistentes no código.
Critério de validação	Inspeção estática dos arquivos do repositório, sem alteração ou execução do sistema.

2. Resumo executivo

O **The Barber** é uma aplicação web renderizada no servidor para cadastro e autenticação de clientes, exibição pública de serviços e consulta dos agendamentos vinculados ao cliente autenticado. O projeto utiliza Node.js como ambiente de execução, Express para o servidor HTTP e as rotas, Handlebars para geração das páginas, Sequelize como ORM e MySQL como banco de dados. O Node.js permite executar JavaScript fora do navegador e oferece primitivas assíncronas de entrada e saída; o Express organiza endpoints por métodos HTTP; o Sequelize fornece mapeamento objeto-relacional; e o Handlebars combina templates com objetos para gerar HTML.[\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)

A autenticação implementada usa **JSON Web Token (JWT)** armazenado em cookie `httpOnly`, com validade de uma hora. As senhas são protegidas com hash `bcryptjs`. Apesar de o servidor também configurar `express-session` para mensagens flash, a identidade do usuário não é mantida na sessão; ela é recuperada a partir do JWT. Essa distinção é importante porque parte da documentação já existente no repositório descreve uma versão anterior baseada em `req.session.userId`, que não corresponde ao código atual.

O núcleo disponível permite cadastrar cliente, autenticar, encerrar a sessão, listar serviços e consultar agendamentos do cliente autenticado. A criação de agendamentos, a recuperação de senha, o contato e o relacionamento formal entre barbeiros e agendamentos ainda não estão implementados. Portanto, o sistema deve ser entendido como um **MVP em desenvolvimento**, com autenticação funcional e módulo de agenda apenas parcialmente concluído.

3. Visão geral do produto

3.1 Problema atendido

O sistema pretende centralizar a interação entre uma barbearia e seus clientes. A versão atual reduz o esforço de apresentação dos serviços e estabelece a base de identificação do cliente e consulta de agenda. O objetivo futuro evidente no código é permitir que o próprio cliente selecione serviços e marque um horário com um barbeiro.

3.2 Objetivo geral

Disponibilizar uma aplicação web simples para que clientes conheçam os serviços oferecidos, criem uma conta, autentiquem-se e acompanhem seus agendamentos.

3.3 Objetivos específicos

Objetivo	Situação no código
Exibir serviços cadastrados no banco	Implementado
Permitir cadastro de cliente	Implementado
Proteger senhas com hash	Implementado
Permitir login por telefone e senha	Implementado
Manter autenticação por JWT em cookie	Implementado
Exibir agendamentos do cliente autenticado	Implementado, dependente de dados previamente inseridos
Permitir criar agendamento pela interface	Não implementado
Recuperar senha por telefone/código	Interface sem backend
Permitir gestão por barbeiro ou administrador	Não implementado

3.4 Perfis e atores

O código reconhece dois estados de interação. O **Visitante** pode acessar a lista de serviços, abrir o cadastro e realizar login. O **Cliente autenticado** possui as mesmas capacidades públicas e também pode consultar seus agendamentos e efetuar logout. Embora Barbeiro seja uma entidade persistida, não existe autenticação, painel ou rota que permita sua interação direta. Não há perfil de administrador implementado; o nome da pasta views/admin não representa um papel de acesso real.

Ator	Responsabilidade	Evidência
Visitante	Consultar serviços, cadastrar-se e autenticar-se	Rotas públicas em <code>authRoutes.js</code> e GET <code>/</code>
Cliente	Consultar agenda própria e encerrar autenticação	Middleware <code>checkToken</code> em GET <code>/agendamento</code> e rota <code>/logout</code>

Ator	Responsabilidade	Evidência
Barbeiro	Entidade de domínio ainda sem interação com o sistema	Model <code>Barbeiro.js</code> sem associações ou rotas
Administrador	Papel não existente	Ausência de model, credenciais, autorização e rotas administrativas

4. Escopo e estado de implementação

4.1 Funcionalidades disponíveis

Código	Funcionalidade	Estado	Observação
RF-01	Listar serviços	Implementada	Consulta todos os registros de <code>Servicos</code> e renderiza a página inicial.
RF-02	Exibir formulário de cadastro	Implementada	Rota pública <code>GET /cadastro</code> .
RF-03	Cadastrar cliente	Implementada	Verifica confirmação da senha e telefone duplicado, gera hash e cria cliente.
RF-04	Exibir formulário de login	Implementada	Rota pública <code>GET /login</code> .
RF-05	Autenticar cliente	Implementada	Busca por telefone, compara hash e grava JWT em cookie.
RF-06	Disponibilizar usuário às views	Implementada	<code>checkUser</code> popula <code>res.locals.user</code> quando o token é válido.
RF-07	Proteger consulta de agendamentos	Implementada	<code>checkToken</code> valida token e carrega <code>req.user</code> .
RF-08	Listar agendamentos do cliente	Implementada	Faz consulta com <code>Cliente</code> , <code>Agendamento</code> e <code>Servicos</code> .
RF-09	Encerrar autenticação	Implementada	Limpa o cookie token e redireciona ao login.

4.2 Funcionalidades parciais ou ausentes

Código	Funcionalidade	Estado	Consequência
GAP-01	Criar agendamento	Não implementada	Método e rota estão comentados; não há gravação de agenda pela interface.
GAP-02	Selecionar serviços	Não implementada	A página lista serviços, mas não contém formulário funcional de seleção.

Código	Funcionalidade	Estado	Consequência
GAP-03	Selecionar barbeiro	Não implementada	Não há listagem, associação ORM ou campo de interface funcional.
GAP-04	Recuperar senha	Interface sem backend	Link /recuperar e formulário existem, mas não há rota GET ou POST correspondente.
GAP-05	Página de contato	Link sem rota	O menu autenticado aponta para /contato, que retorna 404.
GAP-06	Administração	Não implementada	Não existem papéis, autorização ou operações de gestão.
GAP-07	Estado vazio da agenda	Parcial	O controller calcula <code>emptyAgendamento</code> , mas a view não apresenta mensagem específica.
GAP-08	Testes automatizados	Ausente	O script <code>npm test</code> encerra com erro proposital.

5. Requisitos funcionais

5.1 RF-01 — Consultar serviços

O sistema deve permitir que qualquer visitante acesse a raiz / e visualize os serviços cadastrados. O controller executa `Servicos.findAll({ raw: true })` e entrega o resultado à view `agendamento/servico.handlebars`.

5.2 RF-02 — Cadastrar cliente

O sistema deve receber nome, telefone, senha e confirmação de senha. As duas senhas devem ser iguais e o telefone não pode estar associado a outro cliente. A senha é convertida em hash com salt de custo 10 antes da persistência. Após o cadastro, o sistema cria um JWT e autentica o cliente automaticamente.

5.3 RF-03 — Autenticar cliente

O cliente deve informar telefone e senha. O sistema localiza o registro pelo telefone e compara a senha informada com o hash persistido. Quando as credenciais são válidas, o servidor cria um JWT com o identificador do cliente e validade de uma hora, grava o token em cookie `httpOnly` e redireciona para a página inicial.

5.4 RF-04 — Identificar usuário nas páginas

Em cada requisição, o middleware `checkUser` tenta validar o cookie de autenticação. Quando o token e o cliente são válidos, o objeto do cliente é disponibilizado em `res.locals.user`, permitindo que o layout altere o menu de navegação.

5.5 RF-05 — Consultar agendamentos próprios

A rota GET /agendamento deve ser acessível somente com JWT válido. O sistema consulta o cliente identificado e inclui seus agendamentos e os serviços associados a cada um. A view exibe identificador, data, hora, status e a lista de serviços com nome e preço.

5.6 RF-06 — Efetuar logout

Ao acessar /logout, o sistema deve remover o cookie token e redirecionar o navegador para /login.

6. Regras de negócio

Código	Regra	Implementação atual
RN-01	A confirmação de senha deve coincidir com a senha.	Comparação direta no cadastro.
RN-02	O telefone deve identificar unicamente o cliente no fluxo de cadastro.	Consulta prévia por telefone; falta índice <code>unique</code> no model.
RN-03	A senha não deve ser armazenada em texto aberto.	Hash com <code>bcryptjs</code> , salt de custo 10.
RN-04	O cliente deve ser autenticado após cadastro bem-sucedido.	JWT criado após <code>Cliente.create</code> .
RN-05	O login deve usar telefone e senha.	Consulta por telefone e <code>bcrypt.compareSync</code> .
RN-06	Mensagens de login não devem revelar qual credencial falhou.	Mesma mensagem para usuário inexistente e senha incorreta.
RN-07	O token de autenticação deve expirar após uma hora.	<code>expiresIn: '1h'</code> e cookie com <code>maxAge: 3600000</code> .
RN-08	Apenas o cliente autenticado pode consultar sua agenda.	Rota protegida por <code>checkToken</code> ; consulta usa <code>req.user.id</code> .
RN-09	Cada agendamento pertence a um cliente.	Associação <code>Cliente.hasMany</code> e <code>Agendamento.belongsTo</code> .
RN-10	Um agendamento pode conter vários serviços e um serviço pode estar em vários agendamentos.	Relação muitos-para-muitos por <code>ServicoAgendamento</code> .
RN-11	Um agendamento deve possuir data, hora e status.	Campos com <code>allowNull: false</code> .
RN-12	Logout encerra a autenticação no navegador.	Remoção do cookie token.

7. Requisitos não funcionais

Os requisitos abaixo combinam características observadas e necessidades recomendadas para tornar o sistema adequado a um ambiente real.

Código	Categoria	Requisito	Estado
RNF-01	Segurança	Senhas devem ser persistidas por hash.	Atendido
RNF-02	Segurança	Cookie de autenticação deve impedir acesso por JavaScript.	Atendido com <code>httpOnly</code>
RNF-03	Segurança	Cookie deve usar <code>secure: true</code> em HTTPS e política <code>sameSite</code> .	Não atendido
RNF-04	Configuração	Segredos e credenciais devem vir de variáveis de ambiente.	Atendido no código; falta <code>.env.example</code>
RNF-05	Manutenibilidade	Código deve ser separado em rotas, controllers, models e views.	Atendido
RNF-06	Confiabilidade	Erros assíncronos devem gerar respostas controladas.	Parcial
RNF-07	Qualidade	Fluxos críticos devem possuir testes automatizados.	Não atendido
RNF-08	Banco de dados	Alterações estruturais devem usar migrations.	Não atendido; usa <code>sequelize.sync()</code>
RNF-09	Observabilidade	Eventos e falhas devem usar logs estruturados sem dados sensíveis.	Não atendido
RNF-10	Usabilidade	Formulários devem validar campos obrigatórios no cliente e no servidor.	Parcial
RNF-11	Acessibilidade	Páginas devem possuir labels, estrutura semântica e navegação por teclado.	Parcial
RNF-12	Compatibilidade	Interface deve adaptar-se a diferentes larguras de tela.	Há CSS responsivo, mas requer validação visual

8. Arquitetura do sistema

8.1 Estilo arquitetural

O projeto segue uma organização inspirada em **MVC**. As rotas representam a entrada HTTP, os controllers concentram a coordenação dos fluxos, os models mapeiam dados persistidos e as views produzem HTML. O arquivo `server.js` atua como ponto de composição da aplicação, configurando middlewares, template engine, arquivos estáticos, rotas e inicialização do banco.

8.2 Camadas e responsabilidades

Camada	Arquivos principais	Responsabilidade
Inicialização	<code>server.js</code>	Carregar ambiente, configurar Express e iniciar a aplicação após sincronizar o banco.
Rotas	<code>app/routes/*.js</code>	Mapear método e caminho HTTP para controllers e middlewares.
Controllers	<code>app/controllers/*.js</code>	Orquestrar validações, autenticação, consultas e respostas.
Middlewares	<code>app/middlewares/*.js</code>	Validar JWT, carregar cliente e controlar acesso.
Modelos	<code>app/models/*.js</code>	Definir entidades, campos e associações Sequelize.
Views	<code>app/views/**/*.handlebars</code>	Renderizar telas e dados no servidor.
Persistência	<code>config/database.js</code>	Criar e autenticar conexão Sequelize com MySQL.
Recursos estáticos	<code>public/</code>	Fornecer CSS, JavaScript e imagem de fundo.

8.3 Visão arquitetural

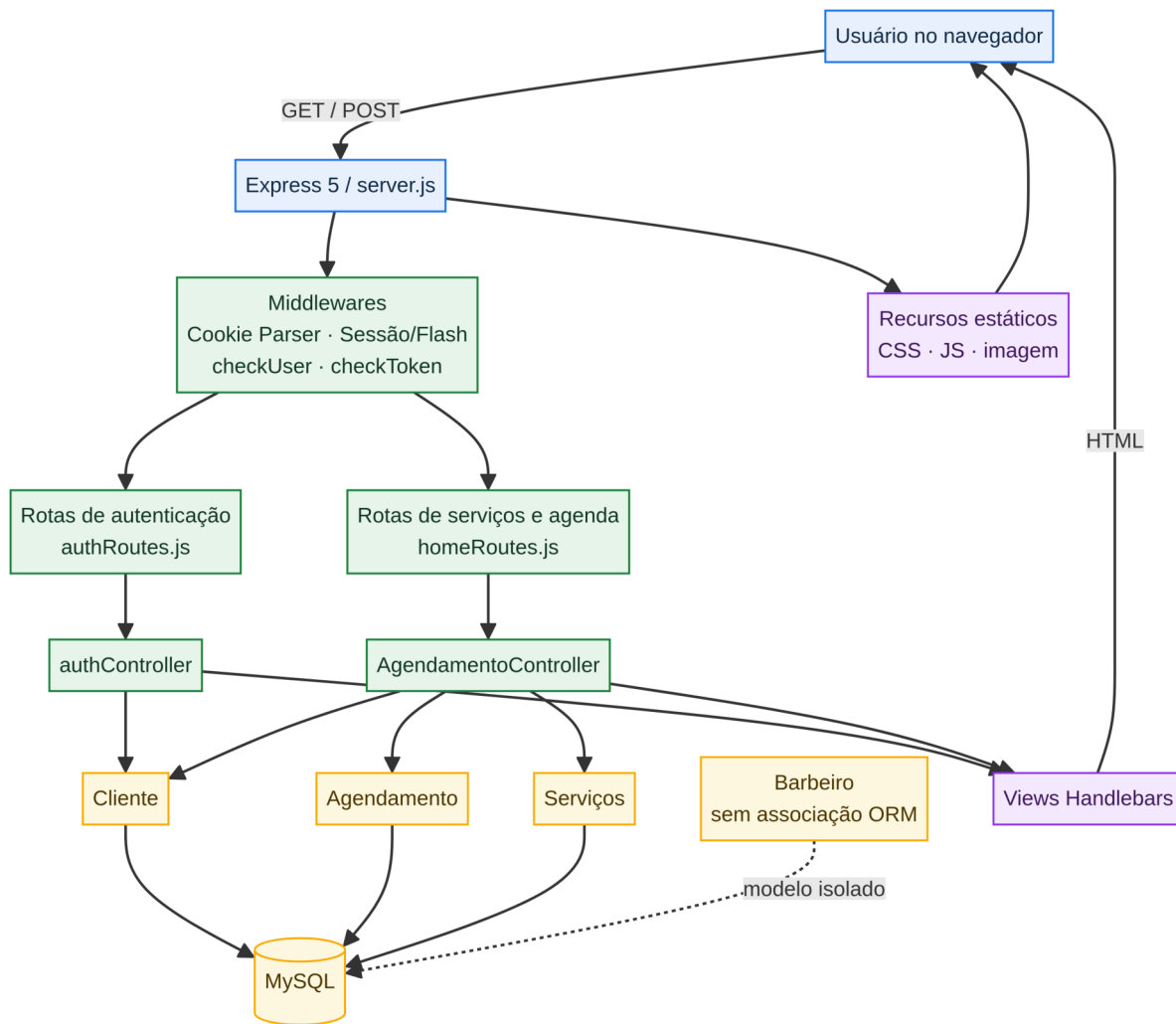


Diagrama de arquitetura do sistema

A figura apresenta o percurso principal entre navegador, Express, middlewares, rotas, controllers, models, MySQL e renderização Handlebars. A linha pontilhada evidencia que o model Barbeiro está persistido, mas permanece isolado das associações ORM.

8.4 Fluxo de uma requisição

Uma requisição chega ao Express, passa pelos parsers de corpo, cookie e sessão, recebe o contexto opcional de usuário pelo checkUser, acessa os arquivos estáticos quando aplicável e, por fim, é encaminhada às rotas. O controller consulta ou altera os models e retorna uma renderização Handlebars ou um redirecionamento HTTP.

8.5 Inicialização

O servidor carrega variáveis com dotenv, configura o Handlebars e os middlewares, registra authRoutes e homeRoutes, executa conn.sync() e começa a escutar na porta definida por PORT. O uso de sincronização automática é conveniente em desenvolvimento, mas migrations são recomendadas para controlar a evolução do esquema.

9. Estrutura de diretórios

```
barbearia-node.js-main/
├── app/
│   ├── controllers/
│   │   ├── agendamentoController.js
│   │   └── authController.js
│   ├── middlewares/
│   │   ├── checkToken.js
│   │   └── checkUser.js
│   ├── models/
│   │   ├── Agendamento.js
│   │   ├── Barbeiro.js
│   │   ├── Cliente.js
│   │   └── Servicos.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   └── homeRoutes.js
│   └── views/
│       ├── admin/agendamentos.handlebars
│       ├── agendamento/agendamento.handlebars
│       ├── agendamento/servico.handlebars
│       ├── auth/login.handlebars
│       ├── auth/recuperar.handlebars
│       ├── auth/register.handlebars
│       └── layouts/main.handlebars
├── config/database.js
├── docs/
├── public/
│   ├── css/
│   ├── imeg/fundo.jpeg
│   └── js/main.js
├── package.json
├── package-lock.json
├── README.md
└── server.js
```

O diretório `public/imeg` provavelmente deveria ser renomeado para `public/img` ou `public/images`, evitando grafia ambígua. O arquivo `public/js/main.js` está vazio, enquanto o layout contém um script inline posicionado antes do corpo; isso pode ocasionar acesso ao DOM antes da criação dos elementos.

10. Tecnologias e dependências

Tecnologia	Versão declarada	Uso no projeto
Express	^5.2.1	Servidor, rotas, middlewares e respostas HTTP
Sequelize	^6.37.7	ORM e associações com MySQL
mysql2	^3.18.2	Driver do banco MySQL
express-handlebars	^9.0.1	Renderização de views

Tecnologia	Versão declarada	Uso no projeto
jsonwebtoken	^9.0.3	Criação e validação de JWT
bcryptjs	^3.0.3	Hash e comparação de senhas
cookie-parser	^1.4.7	Leitura do cookie token
express-session	^1.19.0	Suporte de sessão usado pelas mensagens flash
express-flash	^0.0.2	Mensagens temporárias entre requisições
dotenv	^17.4.2	Carregamento do arquivo .env
nodemon	^3.1.14	Reinício automático em desenvolvimento

As dependências `body-parser`, `cors`, `ejs` e `session-file-store` estão declaradas no `package.json`, mas não são importadas pelo código atual. Elas devem ser removidas se não houver uso planejado, reduzindo superfície de manutenção.

11. Rotas e contratos HTTP

A aplicação é server-rendered e não constitui uma API REST. Os formulários usam `application/x-www-form-urlencoded`; as respostas principais são HTML e redirecionamentos.

Método	Caminho	Controller	Acesso	Resultado
GET	/	<code>AgendamentoController.servicos</code>	Público	Renderiza a lista de serviços.
GET	/login	<code>authController.login</code>	Público	Renderiza o formulário de login.
POST	/login	<code>authController.loginPost</code>	Público	Valida credenciais, cria cookie e redireciona.
GET	/cadastro	<code>authController.register</code>	Público	Renderiza o formulário de cadastro.
POST	/cadastro	<code>authController.registerPost</code>	Público	Valida e cria cliente, cria cookie e redireciona.
GET	/logout	<code>authController.logout</code>	Sem middleware explícito	Limpa o cookie e redireciona para login.
GET	/agendamento	<code>AgendamentoController.dashboard</code>	JWT obrigatório	Renderiza a agenda do cliente.

11.1 POST `/login`

Campo	Tipo	Obrigatório na interface	Tratamento no backend
-------	------	--------------------------	-----------------------

Campo	Tipo	Obrigatório na interface	Tratamento no backend
telefone	Texto/telefone	Sim	Busca Cliente por igualdade.
senha	Texto	Sim	Comparada com o hash por <code>bcrypt.compareSync</code> .

Em caso de sucesso, o servidor grava token com `httpOnly: true`, `secure: false` e validade de 3.600.000 milissegundos. Em falha, utiliza uma mensagem genérica e redireciona novamente para `/login`.

11.2 POST `/cadastro`

Campo	Tipo	Obrigatório na interface	Tratamento no backend
nome	Texto	Não marcado como required	Persistido diretamente.
telefone	Texto/telefone	Não marcado como required	Verificado para duplicidade antes da criação.
senha	Texto	Não marcado como required	Convertido em hash.
confirma_senha	Texto	Não marcado como required	Comparado com senha.

O backend não valida formato, tamanho ou ausência dos campos. O model `Cliente` utiliza as opções incorretas `allowNull` e `require`, que não equivalem a `allowNull: false` ou `validate`. Como resultado, as restrições pretendidas podem não ser aplicadas pelo Sequelize.

11.3 GET `/agendamento`

A rota executa o middleware `checkToken`. Após a validação, o controller consulta o cliente pelo identificador do token e inclui `Agendamento` e `Servicos`. O código pressupõe que o cliente exista e chama `cliente.agendamentos`; a ausência inesperada de registro pode provocar exceção. Logs de dados do cliente e da agenda são enviados ao console e devem ser removidos ou anonimizados em produção.

11.4 Rotas inexistentes referenciadas pela interface

Caminho	Origem	Problema
GET <code>/recuperar</code>	Link na tela de login	Não há rota para renderizar a view.
POST <code>/recuperar</code>	Formulário de recuperação	Não há processamento de código ou nova senha.
GET <code>/contato</code>	Menu do usuário autenticado	Não há rota ou view registrada.
POST <code>/agendamento</code>	Formulário vazio e código comentado	A rota e o método de criação estão comentados.

12. Autenticação e controle de acesso

12.1 Cadastro e login

No cadastro, o servidor valida igualdade das senhas, impede duplicidade lógica de telefone, cria o hash e persiste o cliente. No login, busca o registro por telefone e valida a senha. Nos dois fluxos bem-sucedidos, o JWT contém apenas { id: cliente.id }, é assinado por JWT_SECRET e expira em uma hora.

12.2 Cookies e sessão

Há dois mecanismos distintos. O cookie token mantém a autenticação e é lido por cookie-parser. A sessão do Express é utilizada pelo express-flash para mensagens temporárias. A sessão não armazena a identidade do cliente no código atual. O cookie da sessão tem httpOnly e uma hora de duração, enquanto o cookie JWT também tem httpOnly, mas está configurado com secure: false.

12.3 Middlewares

checkToken exige um token, verifica assinatura e validade, carrega o cliente, popula req.user e interrompe o acesso quando algo falha. checkUser é não bloqueante: tenta recuperar o cliente e o disponibiliza em res.locals.user, mas continua a requisição mesmo sem autenticação. Essa separação permite distinguir **proteção de rota** de **personalização visual**.

12.4 Recomendações de segurança

Em produção, o cookie JWT deve usar secure: true, sameSite: 'lax' ou política mais restritiva, e opções coerentes com HTTPS. O sistema deve usar segredos longos, rotação de chaves, limitação de tentativas de login, cabeçalhos de segurança, validação de entrada, proteção CSRF e mensagens de erro controladas. O telefone deve receber índice único no banco, pois a verificação em código não impede condições de corrida.

13. Modelo de dados

13.1 Entidades

Entidade	Campos explícitos	Campos implícitos do Sequelize	Observação
Cliente	nome, telefone, senha	id, createdAt, updatedAt	Restrições de nulidade estão grafadas incorretamente.
Agendamento	barbeiro_id, data, hora, status	id, clienteId, createdAt, updatedAt	barbeiro_id não possui associação ORM.
Serviço	nome, preco	id, createdAt, updatedAt	Tabela explicitamente nomeada servicos.
Barbeiro	nome	id, createdAt, updatedAt	Sem associações.
ServicoAgendamento	Chaves geradas pelas associações	timestamps conforme configuração padrão	Tabela intermediária automática.

13.2 Relacionamentos

Origem	Cardinalidade	Destino	Implementação
Cliente	1:N	Agendamento	Cliente.hasMany(Agendamento) e Agendamento.belongsTo(Cliente)
Agendamento	N:N	Serviço	belongsToMany nos dois lados, por ServicoAgendamento
Barbeiro	relação pretendida	Agendamento	Apenas campo barbeiro_id; não há belongsTo/hasMany

13.3 Diagrama de classes e entidades

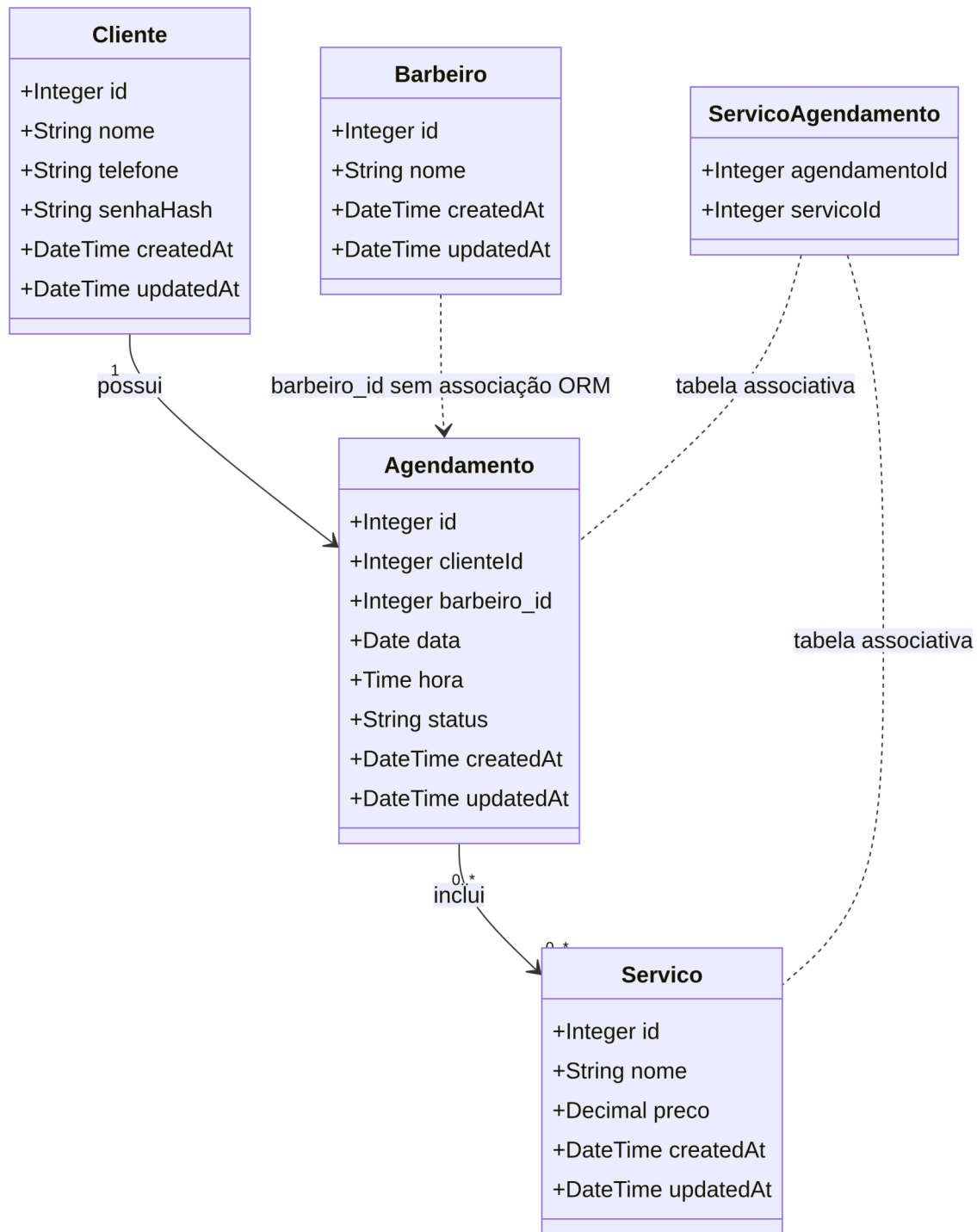


Diagrama de classes e entidades

O diagrama documenta a estrutura persistida e diferencia relações efetivamente declaradas no Sequelize da referência de barbeiro que existe apenas como campo inteiro.

13.4 Dicionário de dados resumido

Tabela	Campo	Tipo lógico	Nulidade esperada	Observação
clientes	id	Inteiro	Não	Chave primária automática.
clientes	nome	Texto	Deveria ser não nulo	<code>allownull</code> deve ser corrigido para <code>allowNull</code> .
clientes	telefone	Texto	Deveria ser não nulo e único	Falta índice único.
clientes	senha	Texto	Deveria ser não nulo	Armazena hash bcrypt.
agendamentos	barbeiro_id	Inteiro	Atualmente opcional	Não é associação Sequelize.
agendamentos	data	Data	Não	<code>DATEONLY</code> .
agendamentos	hora	Hora	Não	<code>TIME</code> .
agendamentos	status	Texto	Não	Falta domínio controlado/enum.
servicos	nome	Texto	Não	Nome do serviço.
servicos	preco	Decimal	Não	Recomenda-se precisão e escala explícitas.
barbeiros	nome	Texto	Não	Único atributo de negócio atual.

13.5 Inconsistências do modelo

O model `Cliente` usa `allownull` com grafia incorreta e `require`, opção que não define obrigatoriedade no Sequelize. O `telefone` não possui restrição `unique`. O `preço` usa `DECIMAL` sem precisão e escala declaradas. O `status` é texto livre, permitindo valores inconsistentes. O `barbeiro` é representado simultaneamente por entidade e por um inteiro solto, sem integridade referencial definida no ORM.

14. Casos de uso

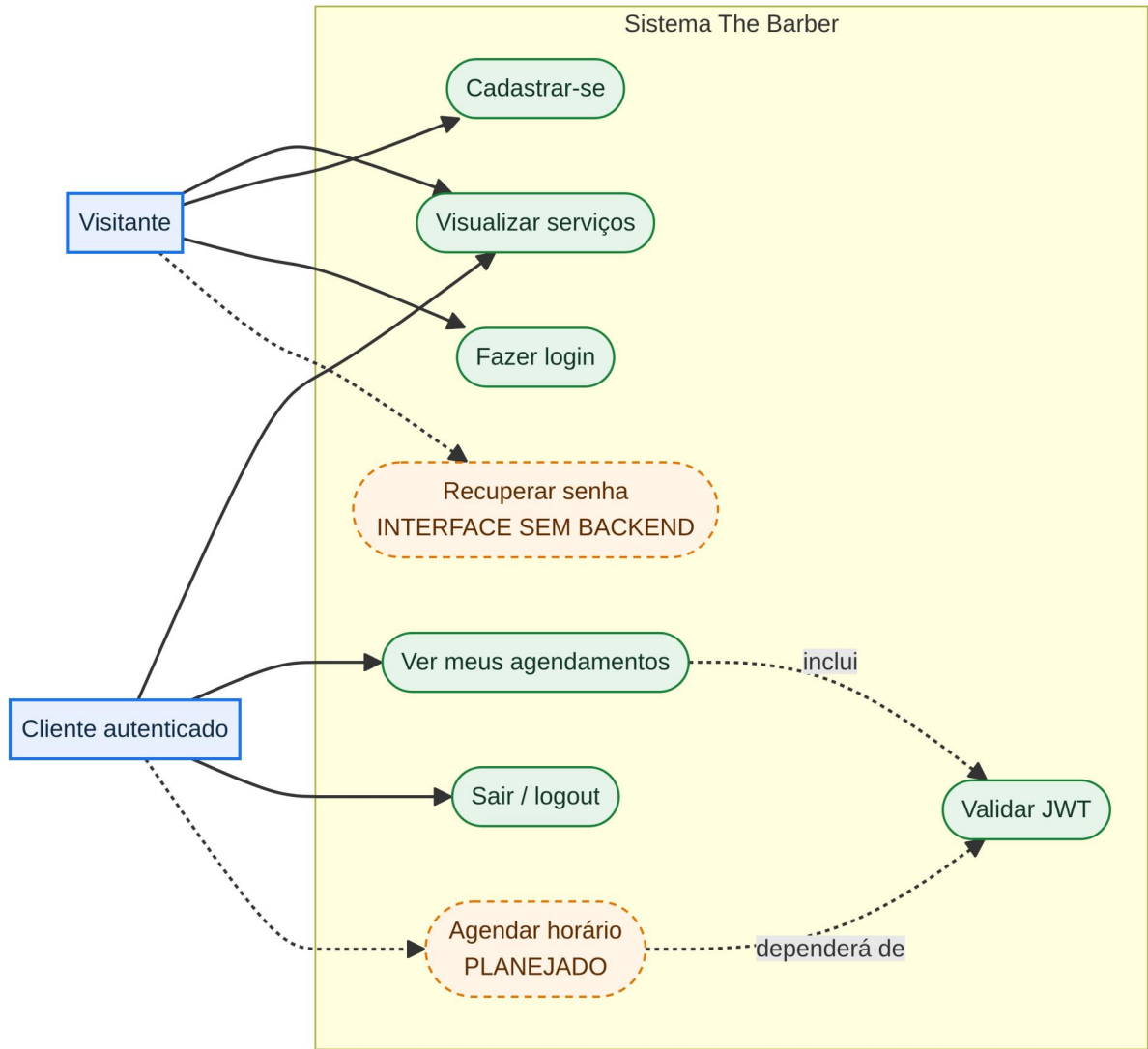


Diagrama de casos de uso

O diagrama diferencia as capacidades implementadas, exibidas em verde, das capacidades planejadas ou sem backend, exibidas com contorno pontilhado em laranja.

14.1 UC-01 — Visualizar serviços

Elemento	Descrição
Ator principal	Visitante ou Cliente
Pré-condição	Aplicação e banco disponíveis
Fluxo principal	O ator acessa /; o sistema consulta serviços; a página apresenta nome e preço.
Pós-condição	Serviços são exibidos sem alterar dados.

Elemento	Descrição
Exceção atual	Não há tratamento local de falha no controller.

14.2 UC-02 — Cadastrar-se

Elemento	Descrição
Ator principal	Visitante
Pré-condição	Telefone ainda não cadastrado
Fluxo principal	Abre cadastro, informa dados, confirma senha, envia; sistema valida, gera hash, cria cliente, emite JWT e redireciona.
Fluxos alternativos	Senhas diferentes; telefone já utilizado; falha de persistência.
Pós-condição	Cliente criado e autenticado.

14.3 UC-03 — Fazer login

Elemento	Descrição
Ator principal	Visitante cadastrado
Pré-condição	Cliente existente
Fluxo principal	Informa telefone e senha; sistema valida, emite JWT e redireciona.
Fluxo alternativo	Credenciais inválidas geram mensagem e retorno à tela de login.
Pós-condição	Cookie de autenticação válido por uma hora.

14.4 UC-04 — Ver meus agendamentos

Elemento	Descrição
Ator principal	Cliente
Pré-condição	JWT válido e cliente existente
Fluxo principal	Acessa /agendamento; middleware valida o token; controller consulta agenda e serviços; view apresenta os resultados.
Fluxo alternativo	Token ausente, inválido ou expirado causa limpeza do cookie e redirecionamento ao login.
Pós-condição	Agenda consultada sem alteração.

14.5 UC-05 — Sair

Elemento	Descrição
Ator principal	Cliente
Pré-condição	Nenhuma exigida pela rota
Fluxo principal	Acessa <code>/logout</code> ; servidor limpa cookie e redireciona para <code>/login</code> .
Pós-condição	Navegador deixa de enviar o JWT anterior.

14.6 UC-06 — Agendar horário (planejado)

Este caso de uso não deve ser considerado funcional. O código contém apenas comentários para um futuro método `agendamentoPost`, uma rota comentada e um formulário vazio. A implementação deverá selecionar cliente autenticado, barbeiro, data, hora e um ou mais serviços, validar disponibilidade e persistir as associações em transação.

15. Fluxos principais

15.1 Fluxo de cadastro

1. O visitante solicita `GET /cadastro`.
2. O servidor renderiza `auth/register`.
3. O visitante envia `POST /cadastro`.
4. O controller compara senha e `confirma_senha`.
5. O controller verifica telefone existente.
6. O servidor gera salt e hash.
7. O cliente é criado no MySQL por meio do Sequelize.
8. Um JWT é emitido e gravado no cookie.
9. O navegador é redirecionado para `/`.

15.2 Sequência de cadastro e login

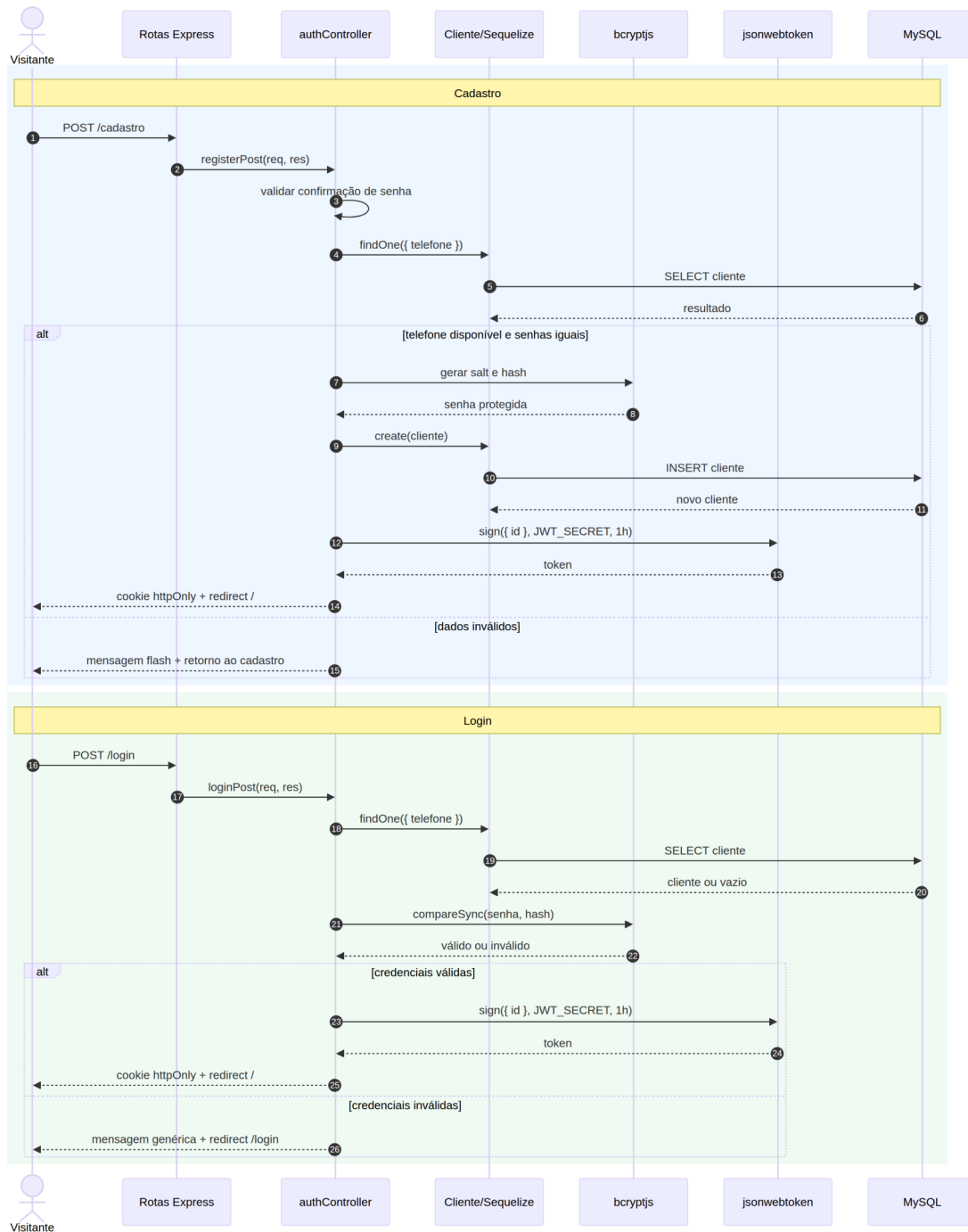


Diagrama de sequência de autenticação

A figura reúne as interações entre navegador, rotas, controller, Sequelize, bcryptjs, JWT e MySQL, incluindo os fluxos alternativos de dados e credenciais inválidos.

15.3 Fluxo de login

10. O visitante solicita `GET /login`.
11. O servidor renderiza o formulário.

12. O visitante envia telefone e senha em POST /login.
13. O controller busca o cliente por telefone.
14. A senha é comparada com o hash persistido.
15. Um JWT é emitido e gravado no cookie.
16. O navegador é redirecionado para /.

15.4 Fluxo de consulta da agenda

17. O cliente solicita GET /agendamento.
18. checkToken lê e valida o JWT.
19. O middleware carrega o cliente e popula req.user.
20. O controller busca o cliente com agendamentos e serviços associados.
21. Os registros são convertidos para objetos simples.
22. A view apresenta data, hora, status e serviços.

15.5 Sequência de consulta protegida da agenda

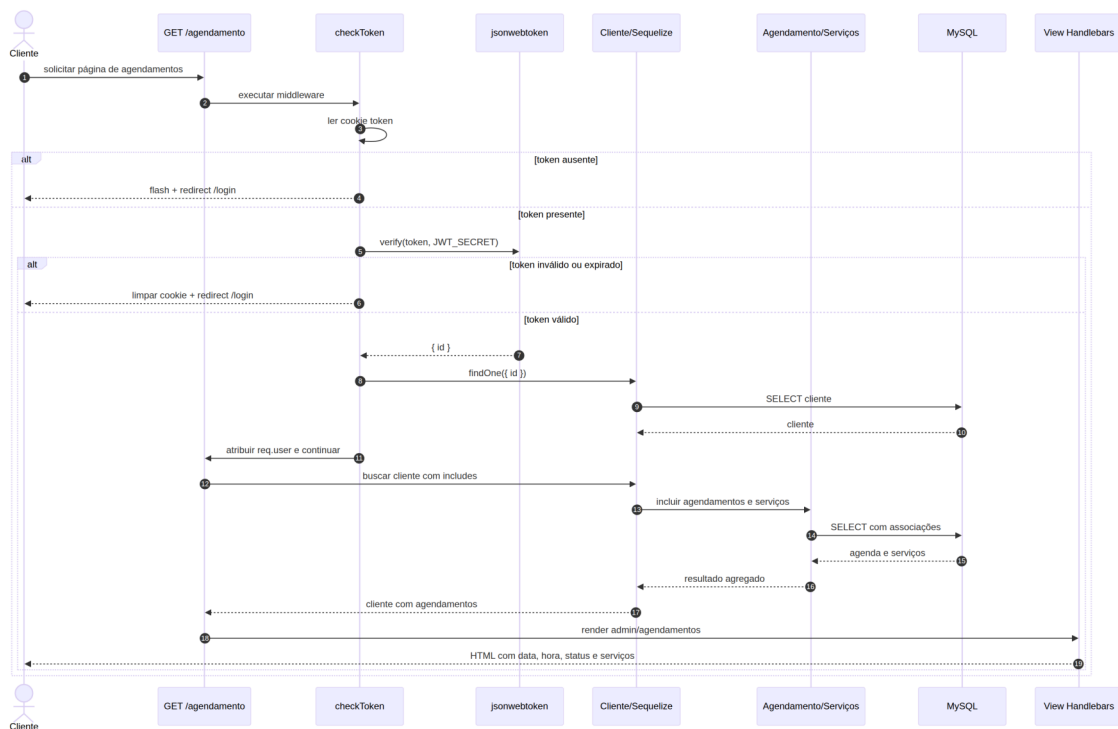


Diagrama de sequência da consulta de agendamentos

A figura destaca as três condições de autenticação: token ausente, token inválido ou expirado e token válido. Somente no terceiro caminho a consulta agregada é executada e a view é renderizada.

16. Instalação e execução local

16.1 Pré-requisitos

É necessário ter Node.js compatível com as dependências do projeto, npm e um servidor MySQL acessível. O banco indicado por DB_NAME deve existir e o usuário deve possuir permissão para criar ou sincronizar tabelas.

16.2 Variáveis de ambiente

Crie um arquivo .env na raiz do projeto. Não versionne valores reais de segredo.

```
PORT=3000
DB_HOST=localhost
DB_NAME=barbearia
DB_USER=usuario_mysql
DB_PASS=senha_mysql
SESSION_SECRET=gere_um_segredo_longo_e_aleatorio
JWT_SECRET=gere_outro_segredo_longo_e_aleatorio
```

16.3 Comandos

```
npm install
npm start
```

Ao iniciar, config/database.js tenta autenticar a conexão e server.js executa sequelize.sync(). O serviço passa a escutar na porta de PORT. Recomenda-se criar um script de desenvolvimento, por exemplo "dev": "nodemon server.js", pois o nodemon está instalado, mas não possui script associado.

16.4 Dados iniciais

O repositório não contém seeders. Para a página inicial exibir conteúdo, devem existir registros na tabela servicos. Para a agenda apresentar resultados, devem existir cliente, agendamentos vinculados e registros na tabela intermediária de serviços. A ausência de fluxo de criação torna necessário inserir dados por script, ferramenta administrativa ou diretamente no banco durante o desenvolvimento.

17. Configuração e operação

Variável	Obrigatória	Finalidade	Risco se ausente
PORT	Sim	Porta HTTP	Servidor pode não iniciar corretamente.
DB_HOST	Sim	Host MySQL	Falha de conexão.
DB_NAME	Sim	Nome do banco	Falha de conexão ou seleção.

Variável	Obrigatória	Finalidade	Risco se ausente
DB_USER	Sim	Usuário MySQL	Falha de autenticação.
DB_PASS	Sim	Senha MySQL	Falha de autenticação.
SESSION_SECRET	Sim	Assinatura da sessão usada pelo flash	Erro ou sessão insegura.
JWT_SECRET	Sim	Assinatura e validação dos tokens	Login e acesso protegido falham; segredo fraco compromete contas.

O processo não implementa endpoint de saúde, desligamento gracioso, retentativa de banco ou tratamento central de erros. Esses itens devem ser adicionados antes de uma implantação com requisitos de disponibilidade.

18. Interface e experiência do usuário

O layout alterna os links de navegação conforme `res.locals.user`. Visitantes visualizam Login e Cadastro; clientes autenticados visualizam Início, Contato, Agendamento e Logout. A página de serviços lista nome e preço; a página de agenda percorre agendamentos e seus serviços; os formulários de login e cadastro enviam dados ao backend.

Há problemas funcionais de interface: o link de recuperação não possui rota GET, o link de contato não possui rota, o formulário de agendamento está vazio, o cadastro não marca os campos como obrigatórios, o estado sem agendamentos não é exibido e há uma tag de fechamento inválida `</1>` no menu. O script de menu está no layout antes do `<body>` e tenta selecionar elementos ainda não construídos, embora `public/js/main.js` esteja vazio.

19. Tratamento de erros e mensagens

Os fluxos de autenticação usam mensagens flash para feedback. O cadastro trata senhas diferentes, telefone duplicado e erro de criação; entretanto, ao falhar no banco, redireciona para `/register`, rota inexistente, em vez de `/cadastro`. Os métodos de listagem não estão envolvidos em `try/catch`, e não há middleware global de erro. Logs com `cliente.toJSON()` podem expor dados pessoais e hash de senha no console.

Problema	Severidade	Correção recomendada
Redirecionamento para <code>/register</code> inexistente	Alta	Alterar para <code>/cadastro</code> .
Log do objeto completo do cliente	Alta	Remover ou registrar apenas identificadores não sensíveis.
Falta de tratamento em consultas	Alta	Criar wrapper assíncrono e middleware global de erro.
Falta de página 404	Média	Adicionar handler final para rota não encontrada.

Problema	Severidade	Correção recomendada
Mensagens sem padronização	Baixa	Centralizar mensagens e corrigir ortografia.

20. Qualidade, testes e validação

O projeto não possui suíte de testes. O script test do package.json apenas informa que nenhum teste foi especificado e retorna código de erro. A estratégia recomendada deve cobrir unidades de validação, integração de rotas e persistência em banco de teste.

Nível	Casos prioritários
Unitário	Comparação de senhas, geração/validação de token, validações de cadastro.
Integração	Cadastro, login, token ausente/expirado, consulta de agenda e logout.
Banco	Associações Cliente–Agendamento e Agendamento–Serviços, unicidade do telefone.
Interface	Formulários, navegação condicional, estado vazio e responsividade.
Segurança	CSRF, cookies, rate limiting, entradas malformadas e exposição de dados.

20.1 Critérios mínimos de aceite

O cadastro deve rejeitar campos vazios, senhas diferentes e telefone duplicado. O login deve aceitar apenas credenciais válidas. A rota de agenda deve negar acesso sem token e nunca retornar agenda de outro cliente. O logout deve invalidar o cookie no navegador. Falhas de banco devem resultar em página ou resposta controlada, sem deixar a requisição pendente.

21. Problemas conhecidos e débito técnico

Prioridade	Item	Evidência	Impacto
P0	Criação de agendamento ausente	Método e rota comentados	Impede o objetivo principal do produto.
P0	Validação insuficiente e model Cliente incorreto	<code>allownull</code> e <code>require</code>	Permite dados inválidos.
P0	Logs expõem objeto do cliente	<code>cliente.toJSON()</code>	Risco de privacidade e segurança.
P1	Barbeiro sem associação	<code>barbeiro_id</code> solto	Falta de integridade referencial.
P1	Recuperação e contato retornam 404	Links sem rotas	Jornada quebrada.

Prioridade	Item	Evidência	Impacto
P1	Cookie JWT não seguro para produção	<code>secure: false</code> , sem <code>sameSite</code>	Maior risco em transporte e ataques web.
P1	Sem tratamento global de erros	Ausência de middleware	Falhas inconsistentes.
P1	Sem migrations e seeders	Uso de <code>sync()</code>	Evolução do banco pouco controlada.
P2	Dependências sem uso	<code>body-parser</code> , <code>cors</code> , <code>ejs</code> , <code>session-file-store</code>	Aumenta manutenção.
P2	Sem testes	Script de teste vazio	Regressões não detectadas.
P2	Problemas de HTML/JavaScript	<code></1></code> e script antes do DOM	Menu pode falhar.
P3	README insuficiente	Apenas texto mínimo	Dificulta entrada de colaboradores.

22. Roadmap recomendado

22.1 Etapa 1 — Estabilização

Corrigir o model `Cliente`, criar validação centralizada, remover logs sensíveis, corrigir rotas quebradas, adicionar tratamento de erros e preparar `.env.example`. Adicionar índices e constraints no banco por migrations.

22.2 Etapa 2 — Agendamento funcional

Criar associação entre barbeiro e agendamento, definir disponibilidade, implementar seleção de serviços e barbeiro, validar conflitos de data/hora e persistir todo o fluxo em transação. Apresentar confirmação e estado vazio na interface.

22.3 Etapa 3 — Gestão operacional

Criar papéis e autorização para barbeiros e administradores, agenda geral, cadastro de serviços, gestão de horários, confirmação e cancelamento. Registrar histórico de mudança de status.

22.4 Etapa 4 — Produção e escala

Adicionar testes, observabilidade, proteção de segurança, migrations, CI/CD, contêineres, monitoramento, backups, política de privacidade e estratégia de recuperação de desastre.

Ordem	Entrega	Critério de conclusão
1	Correções críticas	Cadastro e login validados, sem logs sensíveis e sem links quebrados.
2	Banco consistente	Migrations, telefone único e relações completas.

Ordem	Entrega	Critério de conclusão
3	Agendamento ponta a ponta	Cliente cria e consulta agendamento sem intervenção manual.
4	Gestão	Barbeiro/administrador administra agenda com autorização.
5	Qualidade de produção	Testes, segurança, logs e implantação automatizada.

23. Decisões técnicas observadas

A escolha de ES Modules é indicada por "type": "module" no package.json e pelo uso de import/export. O Express 5 organiza as rotas e middlewares. O Sequelize abstrai consultas e cria associações. O Handlebars mantém a renderização no servidor, adequada a um sistema de escopo reduzido. O JWT evita guardar a identidade na sessão do servidor, enquanto a sessão permanece necessária para o express-flash na implementação atual.

Essas decisões são coerentes com um MVP, mas exigem disciplina adicional em validação, erros, migrations e segurança. A presença simultânea de dependências não usadas e documentação antiga baseada em sessão sugere que o projeto passou por mudanças arquiteturais sem limpeza integral do repositório.

24. Glossário

Termo	Definição no contexto do projeto
Controller	Classe que recebe a chamada da rota e coordena validação, banco e resposta.
Cookie httpOnly	Cookie que não pode ser lido por JavaScript do navegador.
Hash	Representação irreversível usada para verificar senhas sem armazená-las em texto aberto.
JWT	Token assinado que carrega o identificador do cliente e possui validade.
Middleware	Função executada durante o processamento da requisição.
Model	Definição Sequelize que representa uma tabela e suas regras de persistência.
MVC	Separação entre modelo, apresentação e controle.
ORM	Camada que mapeia objetos da aplicação para tabelas relacionais.
Rota	Combinação de método HTTP e caminho atendida pela aplicação.
View	Template Handlebars renderizado como HTML.

25. Apêndices

25.1 Matriz de rastreabilidade

Funcionalidade	Rota	Controller/middleware	Model principal	View	Situação
Listar serviços	GET /	AgendamentoController.servicos	Servicos	agendamento/servico	Implementada
Exibir cadastro	GET /cadastro	authController.register	—	auth/register	Implementada
Cadastrar cliente	POST /cadastro	authController.registerPost	Cliente	Redirecionamento	Implementada com lacunas de validação
Exibir login	GET /login	authController.login	—	auth/login	Implementada
Autenticar	POST /login	authController.loginPost	Cliente	Redirecionamento	Implementada
Carregar usuário nas views	Todas as requisições	checkUser	Cliente	Layout principal	Implementada
Proteger agenda	GET /agendamento	checkToken	Cliente	—	Implementada
Consultar agenda	GET /agendamento	AgendamentoController.dashboard	Cliente, Agendamento, Servicos	admin/agendamentos	Implementada para dados existentes
Logout	GET /logout	authController.logout	—	Redirecionamento	Implementada
Recuperar senha	/recuperar	Ausente	Ausente	auth/recuperar	Interface sem backend
Criar agendamento	POST /agendamento	Comentado	Agendamento, Servicos	Formulário vazio	Não implementada
Contato	GET /contato	Ausente	—	Ausente	Não implementada

25.2 Diagrama de objetos

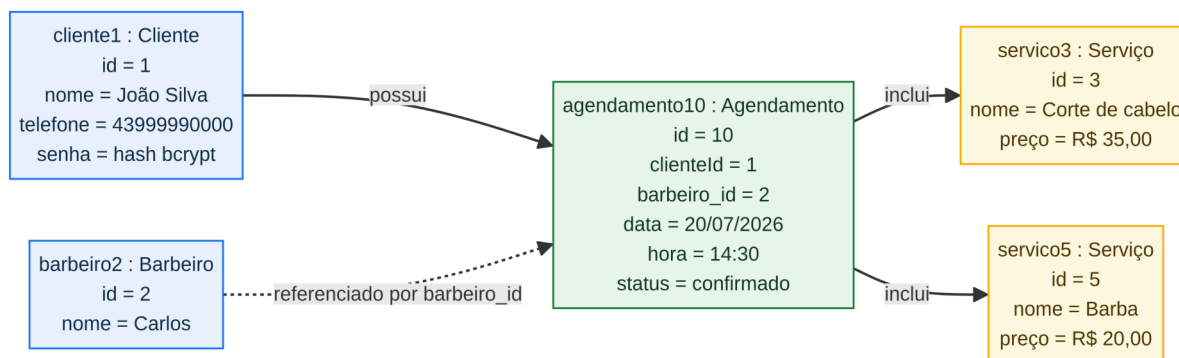


Diagrama de objetos do domínio

O cenário ilustra um cliente com um agendamento confirmado, dois serviços e um barbeiro referenciado pelo campo `barbeiro_id`. Os valores são exemplos documentais e não representam dados extraídos do banco fornecido.

25.3 Checklist para continuidade

Verificação	Situação atual	Ação necessária
Arquivo <code>.env.example</code>	Ausente	Criar modelo sem segredos reais.
Migrations e seeders	Ausentes	Versionar esquema e dados mínimos.
Testes automatizados	Ausentes	Cobrir cadastro, login, token, agenda e logout.
Validação de entrada	Parcial	Adotar schema e mensagens consistentes.
Agendamento ponta a ponta	Ausente	Implementar disponibilidade, transação e associação de serviços.
Associação Barbeiro–Agendamento	Ausente	Definir chaves e associações Sequelize.
Segurança de cookies	Desenvolvimento	Habilitar HTTPS, <code>secure</code> e <code>sameSite</code> em produção.
Tratamento global de erros	Ausente	Adicionar middleware central e página 404.
Logs seguros	Ausente	Remover objetos sensíveis e adotar logs estruturados.
Documentação do README	Insuficiente	Referenciar este documento e incluir início rápido.

26. Referências

- [1] [Node.js — Introduction to Node.js](https://nodejs.org/en/learn/getting-started/introduction-to-nodejs). <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>
- [2] [Express — Routing Guide](https://expressjs.com/en/guide/routing.html). <https://expressjs.com/en/guide/routing.html>
- [3] [Sequelize v6 — Documentation](https://sequelize.org/docs/v6/). <https://sequelize.org/docs/v6/>
- [4] [Handlebars — Guide](https://handlebarsjs.com/guide/). <https://handlebarsjs.com/guide/>